ELEC 271

Section 2 – Logic Circuits

Fall, 2026

Instructor: Dr. Muhammad Saad Asad

Modified from notes provided by Prof. Ryan Grant

Queen's UNIVERSITY

Copyright © Prof. Ryan Grant

ELEC 271

- Moore's Law was first published in 1965
  - Number of transistors on a chip doubles every 18 months

What if it applied to other things?

Then:

- Cars: Oldsmobile Toronado (car of the year) had 385 HP, 0-60mph in 7.5 seconds
- Hydro: Harmon Generating Station: 136 MW

Now?

- Cars: Toronado: HP ~413 Billion HP 0-60mph in 6.9ns
- Hydro: 146 Exa Watts, ~146 028 888 GW

Lecture Outline

- Variables and Functions
- Truth Tables
- Logic Gates and Circuits
- Boolean Algebra
- Logic Synthesis
- NAND and NOR Logic Networks
- Design Examples
- Introduction to CAD tools
- Introduction to VHDL

Read Chapter 2 of the textbook

Introduction to logic circuits

- The study of logic circuits is motivated mostly by their use in digital computers. But such circuits also form the foundation of many other digital systems, such as those that perform control applications or are involved in digital communications. All such applications are based on some simple logical operations that are performed on input information.

- Earlier, we saw that information in computers is represented as electronic signals that can have two discrete values. Although these values are implemented as voltage levels in a circuit, we refer to them simply as logic values, 0 and 1.

- Any circuit in which the signals are constrained to have only some number of discrete values is called a **logic circuit.**

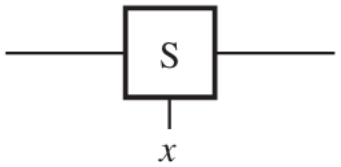
- Logic circuits can be designed with different numbers of logic values, such as three, four, or even more, but we will deal only with **the binary logic circuits** that have two logic values.

Digital Hardware

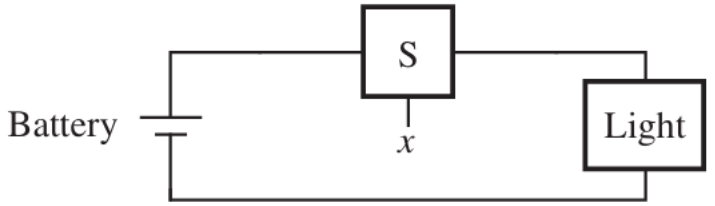
- An analog signal does not rely on discrete values, it can be an essentially infinite number of values (or levels) ex. A radio signal.
- A digital signal is one where there are agreed upon discrete values; for example, a **binary** system where levels are “1” or “0”. In practice an example is a voltage < 0.5V is considered “0”, while a voltage > 2.8V is considered a “1”.
- A switch is a basic element in implementing a digital system.



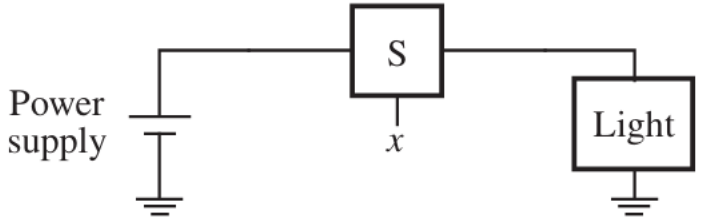
(a) Two states of a switch



(b) Symbol for a switch



(a) Simple connection to a battery



(b) Using a ground connection as the return path

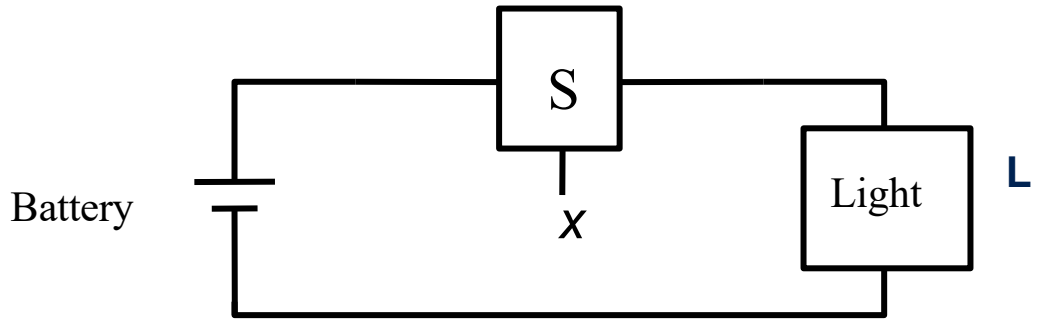
Variables and Functions (Cont'd)

- The state of the light can be defined as a function of input variable x
- The current flows when the switch is closed, that is, when x = 1.

- Logic Function: $L(x) = x$

State of the light:

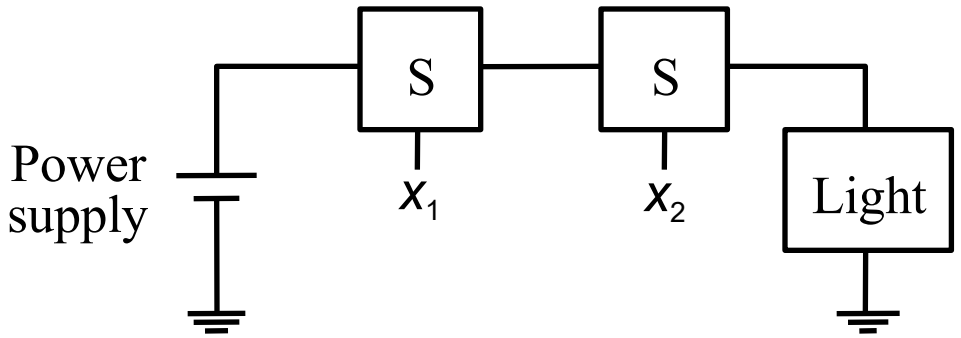
***L* = 1,** light is on, if $x = 1$ (switch closed) ***L* = 0,** light is off, if $x = 0$ (switch open)



Simple connection to a battery

Variables and Functions (Cont'd)

- Two basic functions:

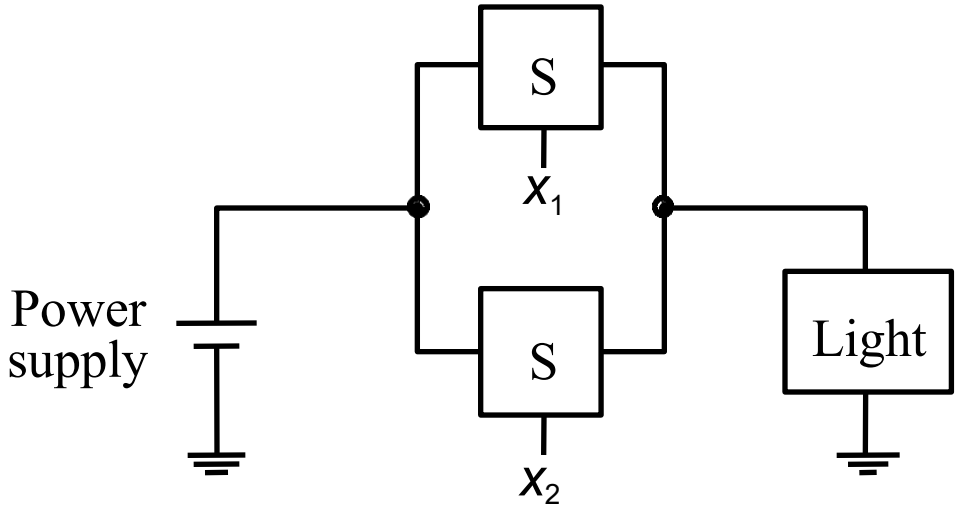


AND operator

or, simply x_1x_2

$L(x_1, x_2) = x_1 . x_2$
where $L = 1$, if $x_1 = 1$ and $x_2 = 1$,
$L = 0$, otherwise

The logical AND function (series connection)



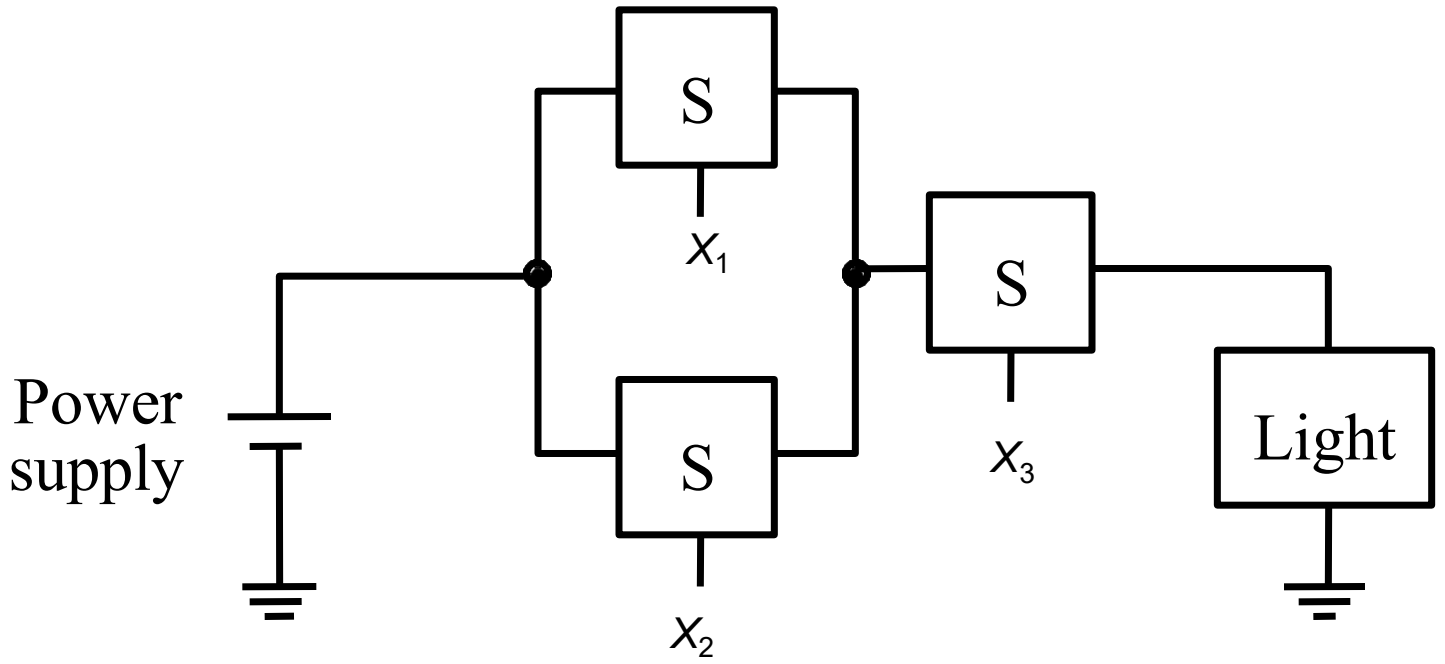
OR operator

$L(x_1, x_2) = x_1 + x_2$
where $L = 1$, if $x_1 = 1$ or $x_2 = 1$
or if $x_1 = x_2 = 1$,
$L = 0$ if $x_1 = x_2 = 0$

The logical OR function (parallel connection)

Variables and Functions (Cont'd)

- A series-parallel connection:



- Logic Function?

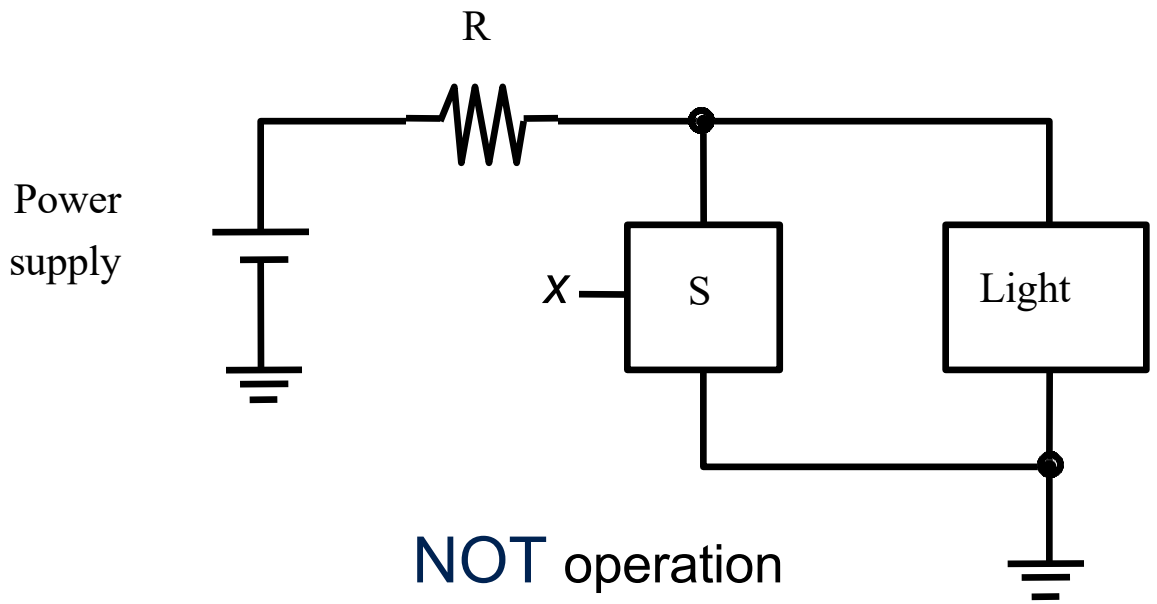
$$L(x_1, x_2, x_3) = (x_1 + x_2) \cdot x_3$$

Variables and Functions (Cont'd)

- Inversion or Complement: a closed switch will short-circuit the light and prevent the current from flowing through it. The light is on when the switch is opened.
  - $L(x) = \overline{x}$ where L = 1 if x = 0
  - L = 0 if x = 1

different notations

$\overline{x} = x' = !x = \sim x = \text{NOT } x$



NOT operation

If $f(x_1, x_2, x_3) = x_1.x_2.x_3$

$\Rightarrow \overline{f}(x_1, x_2, x_3) = \overline{x_1 x_2 x_3}$

Variables and Functions cont'd (Inversion, complement, NOT operation)

$$\bar{x} = x' = !x = \sim x = \text{NOT } x$$

The complement operation can be applied to a single variable or to more complex operations. For example, if

$$f(x_1, x_2) = x_1 + x_2$$

then the complement of f is

$$\bar{f}(x_1, x_2) = \overline{x_1 + x_2}$$

This expression yields the logic value 1 only when neither x_1 nor x_2 is equal to 1, that is, when $x_1 = x_2 = 0$. Again, the following notations are equivalent:

$$\overline{x_1 + x_2} = (x_1 + x_2)' = !(x_1 + x_2) = \sim(x_1 + x_2) = \text{NOT } (x_1 + x_2)$$

Truth Tables

- **Truth table**: is a useful aid for depicting information involving logic functions.
  - For *n* input variables, the truth table has 2^n rows.

All four possible combinations of logic values that of variables x_1 and x_2

x_1	x_2	$x_1 \cdot x_2$	$x_1 + x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1
		AND	OR

Two-input AND and OR operations

Truth Tables (Cont'd)

- Three-input AND and OR operations

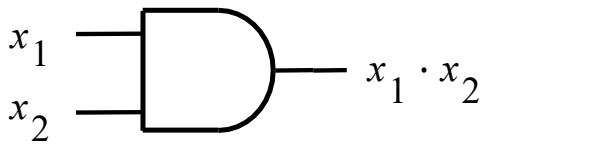
x_1	x_2	x_3	$x_1 \cdot x_2 \cdot x_3$	$x_1 + x_2 + x_3$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A truth table for three input variables (x_1, x_2, x_3) has eight rows because there are eight possible valuations of these variables

Logic Gates and Circuits

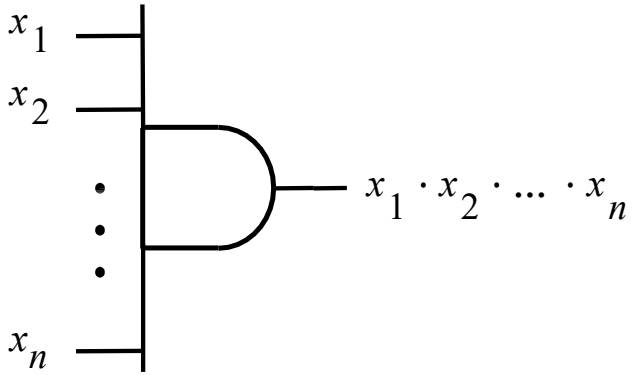
- Basic Gates – graphical symbols:

AND gates

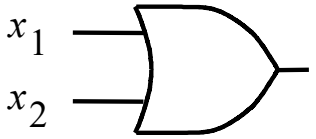


Input: signals x_1, x_2

Output: AND operation $x_1 \cdot x_2$

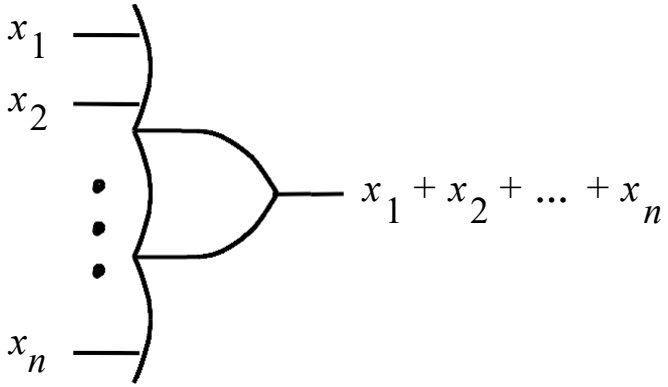


OR gates

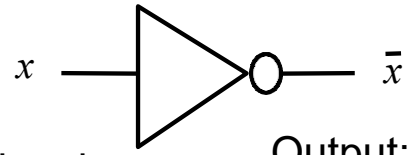


Input: signals x_1, x_2

Output: OR operation $x_1 + x_2$



NOT gate

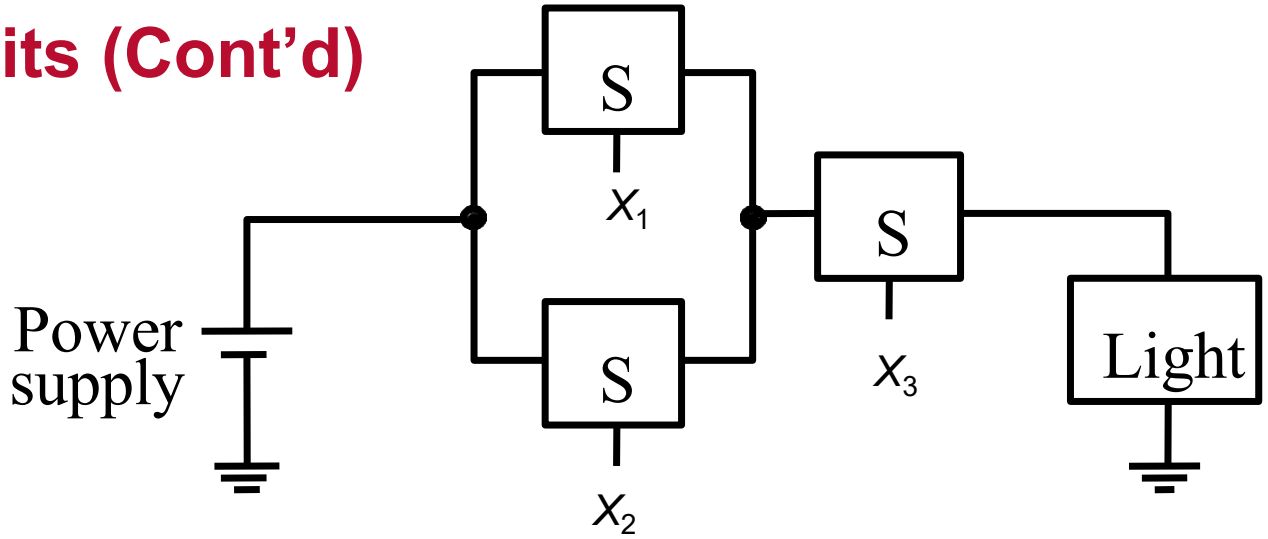


Input: signal x

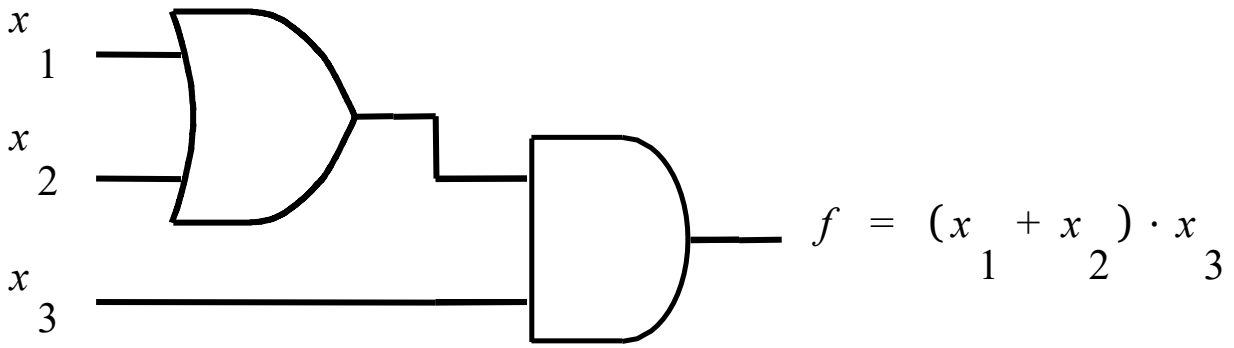
Output: NOT operation $\bar{x}$

Logic Gates and Circuits (Cont'd)

- A logic network or a logic circuit:



- Equivalent logic circuit:

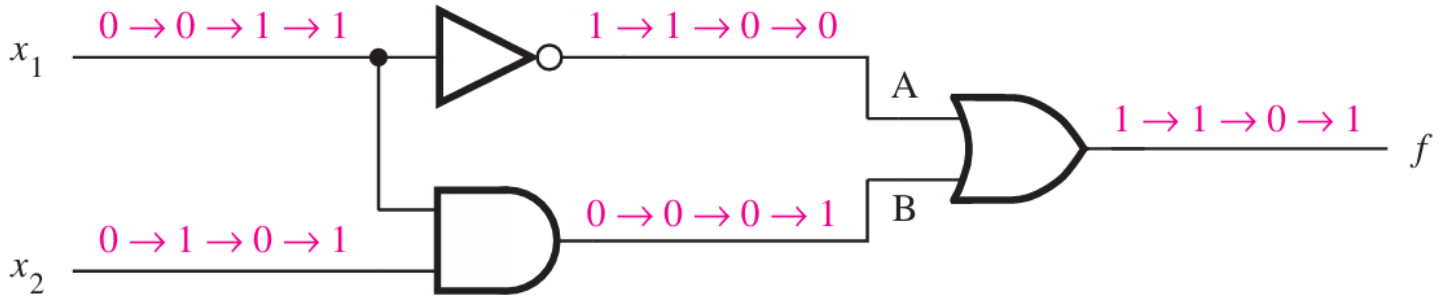


A larger circuit is implemented by a network of gates. In the figure above, the OR operation is applied on signals x_1, x_2. The output of this is then AND'd with signal x_3

Logic Gates and Circuits (Cont'd)

- A designer of digital systems is faced with two basic issues. For an existing logic network, it must be possible to determine the function performed by the network. This task is referred to as the *analysis* process.
- The reverse task of designing a new network that implements a desired functional behavior is referred to as the *synthesis* process.
- The analysis process is rather straightforward and much simpler than the synthesis process.

- The figure below shows a simple network consisting of three gates. To analyze its functional behavior, we can consider what happens if we apply all possible combinations of the input signals to it.



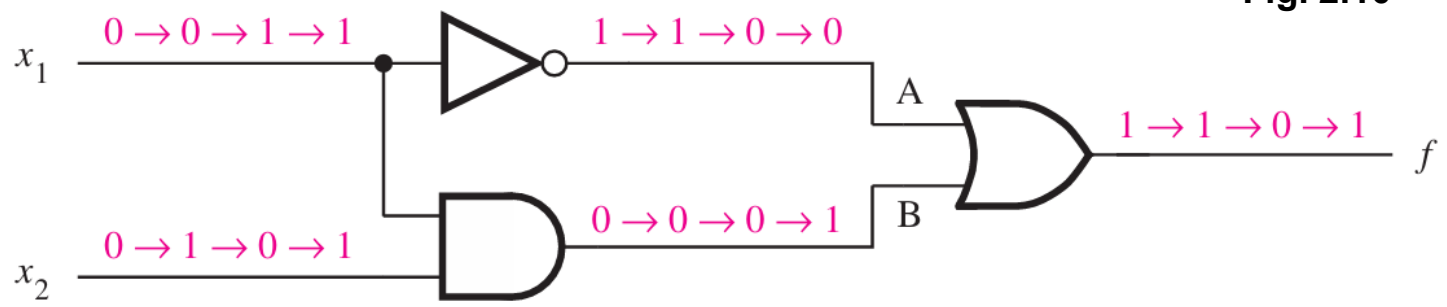
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$

x_1	x_2	$f(x_1, x_2)$	A	B
0	0	1	1	0
0	1	1	1	0
1	0	0	0	0
1	1	1	0	1

(b) Truth table

Logic Gates and Circuits (Cont'd)

Fig. 2.10



(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$

x_1	x_2	$f(x_1, x_2)$	A	B
0	0	1	1	0
0	1	1	1	0
1	0	0	0	0
1	1	1	0	1

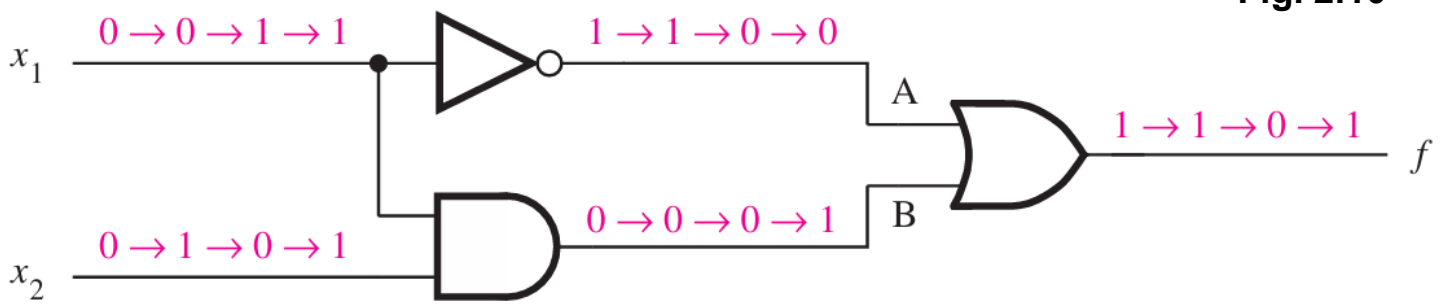
(b) Truth table

Explanation of logic operations being performed:

Figure 2.10*a* shows a simple network consisting of three gates. To analyze its functional behavior, we can consider what happens if we apply all possible combinations of the input signals to it. Suppose that we start by making $x_1 = x_2 = 0$. This forces the output of the NOT gate to be equal to 1 and the output of the AND gate to be 0. Because one of the inputs to the OR gate is 1, the output of this gate will be 1. Therefore, $f = 1$ if $x_1 = x_2 = 0$. If we then let $x_1 = 0$ and $x_2 = 1$, no change in the value of f will take place, because the outputs of the NOT and AND gates will still be 1 and 0, respectively. Next, if we apply $x_1 = 1$ and $x_2 = 0$, then the output of the NOT gate changes to 0 while the output of the AND gate remains at 0. Both inputs to the OR gate are then equal to 0; hence the value of f will be 0. Finally, let $x_1 = x_2 = 1$. Then the output of the AND gate goes to 1, which in turn causes f to be equal to 1. Our verbal explanation can be summarized in the form of the truth table shown in Figure 2.10*b*.

Logic Gates and Circuits (Cont'd)

Fig. 2.10



(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$

x_1	x_2	$f(x_1, x_2)$	A	B
0	0	1	1	0
0	1	1	1	0
1	0	0	0	0
1	1	1	0	1

(b) Truth table

Summary logic operation steps being performed:

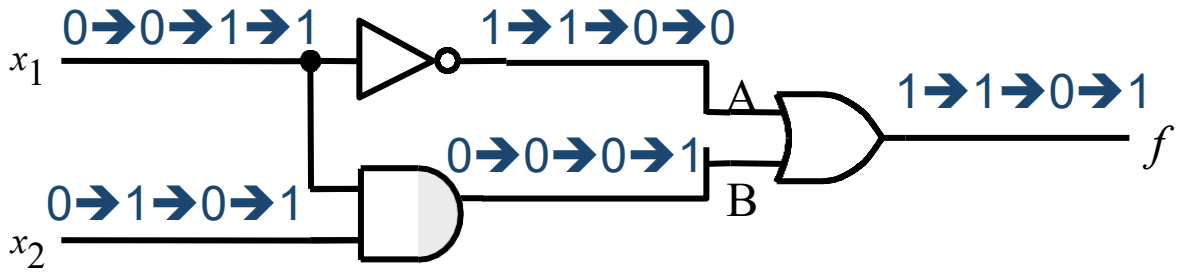
- Output $f(x_1, x_2) = A + B$ (output signal f is the OR operation of signal A and B)
- $A = \bar{x}_1$ (A is NOT operation applied on input x_1)
- $B = x_1 \cdot x_2$ (B is the output to the AND operation applied on signals x_1, x_2)

- Therefore,

$$f(x_1, x_2) = A + B$$
$$= \bar{x}_1 + x_1 \cdot x_2$$

Logic Gates and Circuits (Cont'd)

- Analysis of logic networks:



Network that implements $f = \overline{x_1} + x_1x_2$

Truth table

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

A	B
1	0
1	0
0	0
0	1

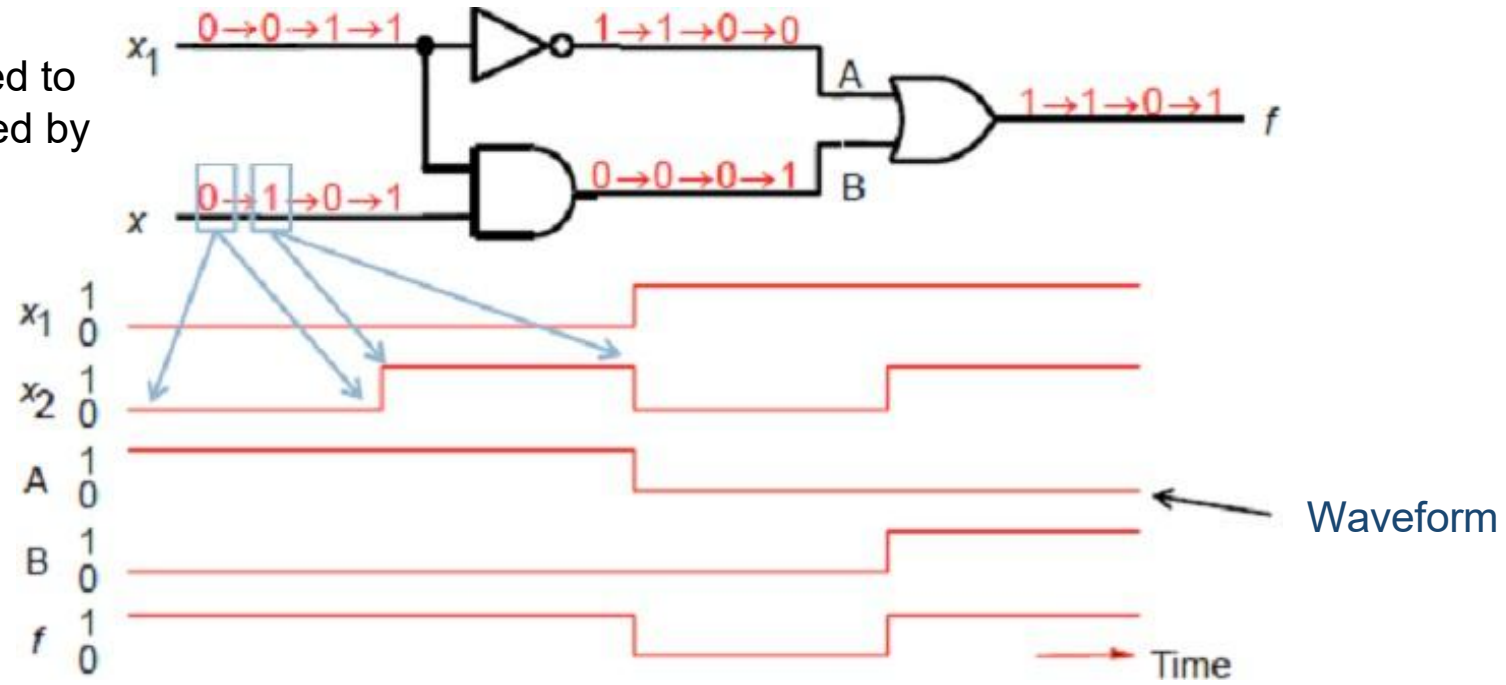
Logic Gates and Circuits (Cont'd)

- Timing Diagram: changes in signals are presented in graphical form.
  - These idealized waveforms are based on the assumption that logic gates respond to changes on their inputs in zero time. However, practical logic gates are implemented using electronic circuits which need some time to change their states

➔ good for indicating functional behaviour of logic circuits.

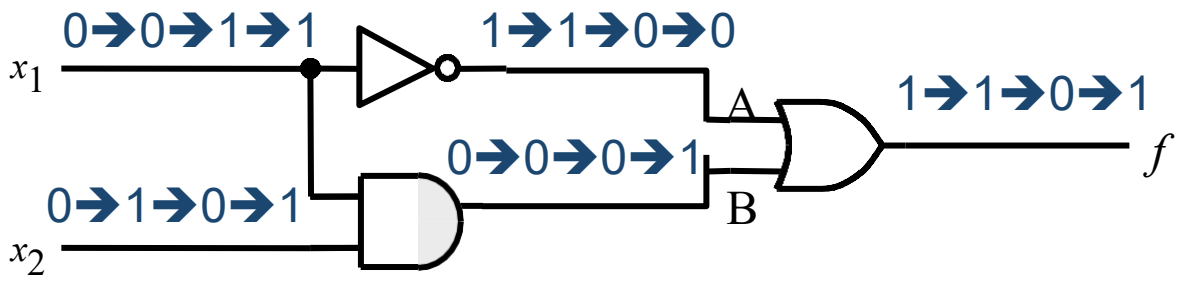
Suppose that the signals x_1 and x_2 are applied to the network in the order (x1,x2) = (0,0) followed by (0,1), (1,0), and (1,1)

The time runs from left to right, and each input valuation is held for some fixed duration. The figures shows the waveforms for the inputs and output of the network, as well as for the internal signals at the points labeled A and B.



Logic Gates and Circuits (Cont'd)

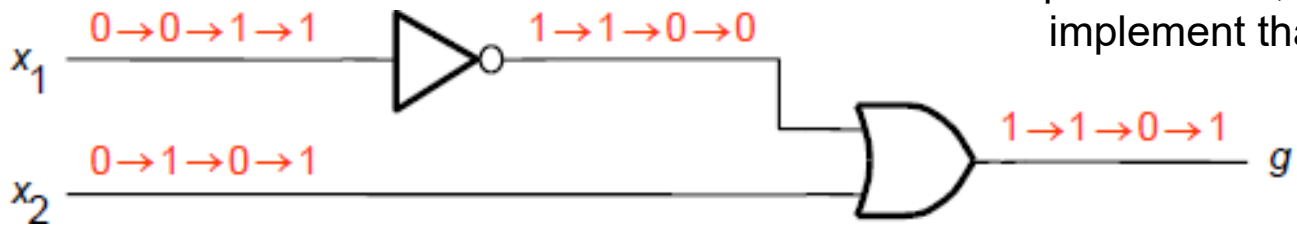
- Functionally equivalent networks: consider this circuit again,



Network that implements $f = \overline{x_1} + x_1x_2$

- Now, consider this one:

simpler network, which is less costly to implement than the above one.



Network that implements $g = \overline{x_1} + x_2$

➔ $g(x_1, x_2) = f(x_1, x_2)$

The output g changes in exactly the same way as f does. Both networks will have the same truth tables.

Logic Gates and Circuits (Cont'd)

- Functionally equivalent networks (cont'd):
  - A logic function can be implemented with a variety of different networks, probably having different costs.

  Question: What is the best implementation for a function?

  - Many techniques exists for synthesizing a logic functions:
    - Through the truth table
    - Main approaches will be discussed in section 4
    - Algebraic or Boolean manipulation of logic expressions. We will discuss this mathematical approach here.

Boolean Algebra

- In 1849, **George Boole** published a scheme for the algebraic description of processes involved in logical thoughts and reasoning.

➔ Boolean algebra

- In 1930, Claude Shannon showed the effectiveness of Boolean algebra for describing circuits built with switches, the work known as switching algebra.
- Boolean algebra is the foundation of our modern digital technology.
  - Can be used to analyze and design logic circuits.

- Boolean algebra :
  - Involves elements that take on one of two values {0, 1};
  - Binary operations are OR (+), and AND (·);
  - A unitary operation NOT ($\bar{}$)

Boolean Algebra (Cont'd)

- Axioms: Boolean algebra is based on a set of rules that are derived from a small number of basic assumptions, called axioms. Assume that the following axioms are true:

1a.	$0 \cdot 0 = 0$	1b.	$1 + 1 = 1$
2a.	$1 \cdot 1 = 1$	2b.	$0 + 0 = 0$
3a.	$0 \cdot 1 = 1 \cdot 0 = 0$	3b.	$1 + 0 = 0 + 1 = 1$
4a.	If $x = 0$ then $\overline{x} = 1$	4b.	If $x = 1$ then $\overline{x} = 0$

- Single-variable theorems: from the axioms, we can define some theorems for dealing with single variables.

5a.	$x \cdot 0 = 0$	5b.	$x + 1 = 1$
6a.	$x \cdot 1 = x$	6b.	$x + 0 = x$
7a.	$x \cdot x = x$	7b.	$x + x = x$
8a.	$x \cdot \overline{x} = 0$	8b.	$x + \overline{x} = 1$
9.	$\overline{\overline{x}} = x$		

  - By perfect induction, that is, by substituting the values $x = 0$ and $x = 1$ into the expressions and using the axioms, one can prove the validity of these theorems.

Boolean Algebra (Cont'd)

- Duality: a dual of a Boolean expression is obtained by replacing all "+" operators with "·" operators, and vice versa, and by replacing all 0s with 1s, and vice versa.

  ➔ this will be clear later that there are at least two different ways to express every logic function with Boolean algebra.

- Two- and Three-variable Properties: if x, y, and z are variables in B, the following properties exist:

10a.	$x \cdot y = y \cdot x$	10b.	$x + y = y + x$	*Commutative*
11a.	$x \cdot (y \cdot z) = (x \cdot y) \cdot z$	11b.	$x + (y + z) = (x + y) + z$	*Associative*
12a.	$x \cdot (y + z) = x \cdot y + x \cdot z$	12b.	$x + y \cdot z = (x + y) \cdot (x + z)$	*Distributive*
13a.	$x + x \cdot y = x$	13b.	$x \cdot (x + y) = x$	*Absorption*
14a.	$x \cdot y + x \cdot \overline{y} = x$	14b.	$(x + y) \cdot (x + \overline{y}) = x$	*Combining*
15a.	$\overline{x \cdot y} = \overline{x} + \overline{y}$	15b.	$\overline{x + y} = \overline{x} \cdot \overline{y}$	*DeMorgan's Thm*
16a.	$x + \overline{x} \cdot y = x + y$	16b.	$x \cdot (\overline{x} + y) = x \cdot y$	
17a.	$x \cdot y + y \cdot z + \overline{x} \cdot z = x \cdot y + \overline{x} \cdot z$	17b.	$(x + y) \cdot (y + z) \cdot (\overline{x} + z) = (x + y) \cdot (\overline{x} + z)$	*Consensus*

Boolean Algebra (Cont'd)

Figure 2.13 illustrates how perfect induction can be used to prove DeMorgan's theorem, using the format of a truth table.

- Proof of DeMorgan's theorem (15a):

Figure 2.13

x	y	$x \cdot y$	$\overline{x \cdot y}$	$\overline{x}$	$\overline{y}$	$\overline{x} + \overline{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0
			LHS			RHS

$\overline{x.y} = \overline{x} + \overline{y}$

Example 2.3 from book

Example 2.3 Let us prove the validity of the logic equation

$$(x_1 + x_3) \cdot (\bar{x}_1 + \bar{x}_3) = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

The left-hand side can be manipulated as follows. Using the distributive property, 12*a*, gives

$$\text{LHS} = (x_1 + x_3) \cdot \bar{x}_1 + (x_1 + x_3) \cdot \bar{x}_3$$

Applying the distributive property again yields

$$\text{LHS} = x_1 \cdot \bar{x}_1 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + x_3 \cdot \bar{x}_3$$

Note that the distributive property allows ANDing the terms in parenthesis in a way analogous to multiplication in ordinary algebra. Next, according to theorem 8*a*, the terms $x_1 \cdot \bar{x}_1$ and $x_3 \cdot \bar{x}_3$ are both equal to 0. Therefore,

$$\text{LHS} = 0 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + 0$$

From 6*b* it follows that

$$\text{LHS} = x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3$$

Finally, using the commutative property, 10*a* and 10*b*, this becomes

$$\text{LHS} = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

which is the same as the right-hand side of the initial equation.

Example 2.4 from book

Example 2.4 Consider the logic equation

$$x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 = \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2$$

The left-hand side can be manipulated as follows

$$\begin{aligned} \text{LHS} &= x_1 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot \bar{x}_3 + \bar{x}_2 \cdot x_3 && \text{using } 10b \\ &= x_1 \cdot (\bar{x}_3 + x_3) + \bar{x}_2 \cdot (\bar{x}_3 + x_3) && \text{using } 12a \\ &= x_1 \cdot 1 + \bar{x}_2 \cdot 1 && \text{using } 8b \\ &= x_1 + \bar{x}_2 && \text{using } 6a \end{aligned}$$

The right-hand side can be manipulated as

$$\begin{aligned} \text{RHS} &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot (x_2 + \bar{x}_2) && \text{using } 12a \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot 1 && \text{using } 8b \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 && \text{using } 6a \\ &= x_1 + \bar{x}_1 \cdot \bar{x}_2 && \text{using } 10b \\ &= x_1 + \bar{x}_2 && \text{using } 16a \end{aligned}$$

Example 2.4 cont'd from book

Being able to manipulate both sides of the initial equation into identical expressions establishes the validity of the equation. Note that the same logic function is represented by either the left- or the right-hand side of the above equation; namely

$$\begin{aligned} f(x_1, x_2, x_3) &= x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2 \end{aligned}$$

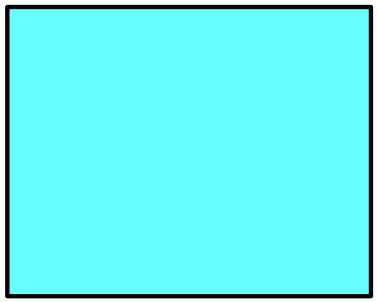
As a result of manipulation, we have found a much simpler expression

$$f(x_1, x_2, x_3) = x_1 + \bar{x}_2$$

which also represents the same function. This simpler expression would result in a lower-cost logic circuit that could be used to implement the function.

The Venn Diagram

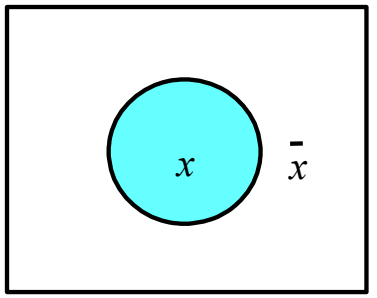
- The Venn diagrams is a visual aid that provides for a more intuitive and understanding of how two expressions may be equivalent. Invented in 1880 by John Venn.



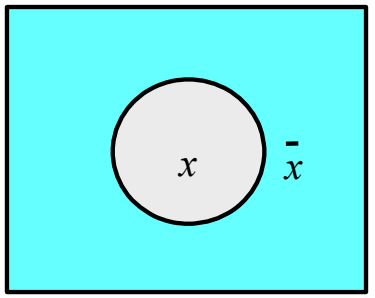
(a) Constant 1



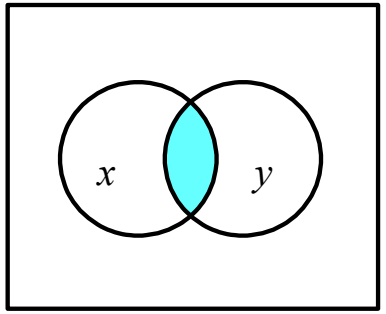
(b) Constant 0



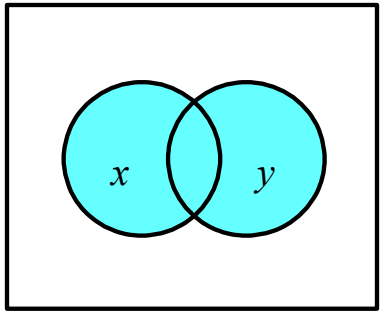
(c) Variable x



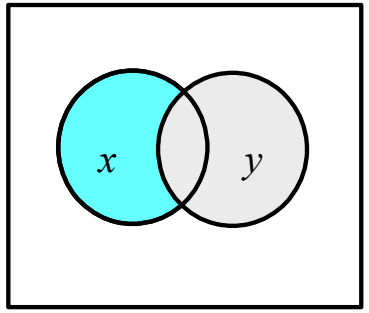
(d) $\bar{x}$



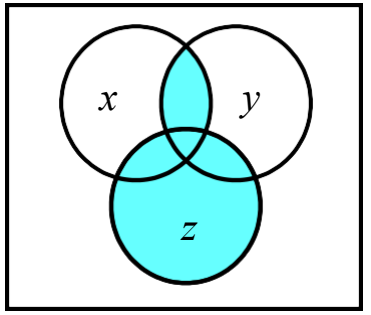
(e) $x \cdot y$



(f) $x + y$



(g) $x \cdot \bar{y}$



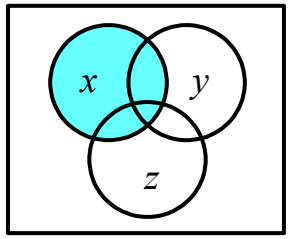
(h) $x \cdot y + z$

Intersection of x and y

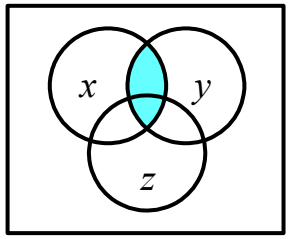
Union of x and y

The Venn Diagram (Cont'd)

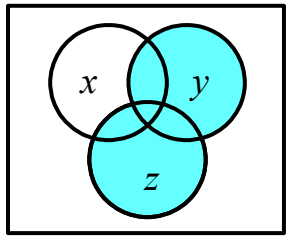
- Verification of the distributive property (12a) – x(y+z) = xy+xz



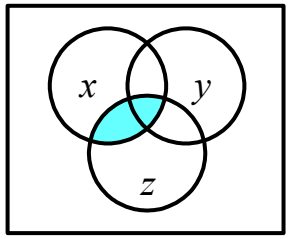
(a) x



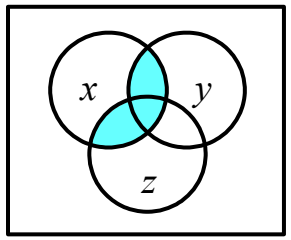
(d) $x \cdot y$



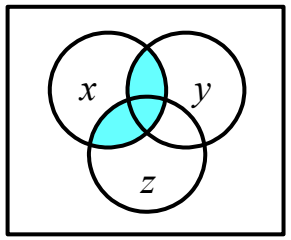
(b) $y+z$



(e) $x \cdot z$



(c) $x \cdot (y+z)$



(f) $x \cdot y + x \cdot z$

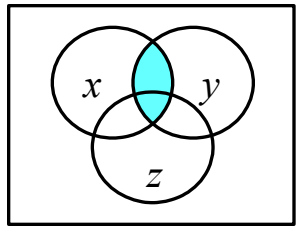
The Venn Diagram (Cont'd)

- Verification example:

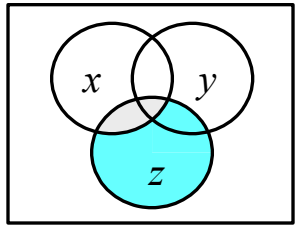
17a: Concensus

$$xy + \bar{x}z + yz = xy + \bar{x}z$$

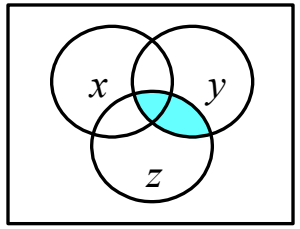
Prove the validity of DeMorgan's theorem using a Venn diagram.



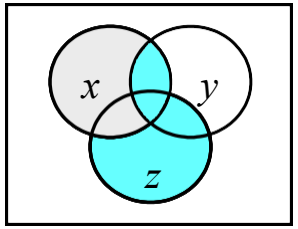
$x \cdot y$



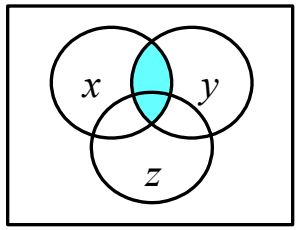
$\bar{x} \cdot z$



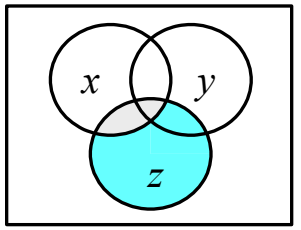
$y \cdot z$



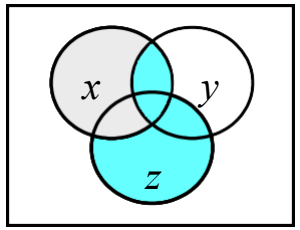
$x \cdot y + \bar{x} \cdot z + \bar{y} \cdot z$



$x \cdot y$



$\bar{x} \cdot z$



$x \cdot y + \bar{x} \cdot z$

Notation and Precedence of Operations

- Logical sum and logical product operations:
  - To avoid confusion between Boolean operators . (AND) and + (OR) with arithmetic multiplication and addition operators, there are two other operators: ^ symbol for AND, and v symbol for OR
  - xy = x·y = x^y x+y = xvy
  - OR operation is also called *logical sum* - AND operation is also called *logical product*
  - You can simply say “sum” and “product”
    - $x_1x_2 + x_2x_3 + x_1x_3$ is a sum of three product terms (SOP).
    - $(x_1 + x_2)\,(x_2 + x_3)\,(x_1 + x_3)$ is a product of three sum terms (POS).

- **Precedence**:
  - Parentheses can force order.
  - In the absence of parentheses, the operations are performed in the order: NOT, AND, and then OR.

Precedence of operation

2.5.3 Precedence of Operations

Using the three basic operations—AND, OR, and NOT—it is possible to construct an infinite number of logic expressions. Parentheses can be used to indicate the order in which the operations should be performed. However, to avoid an excessive use of parentheses, another convention defines the precedence of the basic operations. It states that in the absence of parentheses, operations in a logic expression must be performed in the order: NOT, AND, and then OR. Thus in the expression

$$x_1 \cdot x_2 + \bar{x}_1 \cdot \bar{x}_2$$

it is first necessary to generate the complements of x_1 and x_2. Then the product terms $x_1 \cdot x_2$ and $\bar{x}_1 \cdot \bar{x}_2$ are formed, followed by the sum of the two product terms. Observe that in the absence of this convention, we would have to use parentheses to achieve the same effect as follows:

$$(x_1 \cdot x_2) + ((\bar{x}_1) \cdot (\bar{x}_2))$$

Finally, to simplify the appearance of logic expressions, it is customary to omit the $\cdot$ operator when there is no ambiguity. Therefore, the preceding expression can be written as

$$x_1 x_2 + \bar{x}_1 \bar{x}_2$$

Synthesis using AND, OR, and NOT

- Using the truth table to synthesize a logic function:
  - Derive the truth table
  - Create the product term (valuation) for which the output is 1
  - Logical sum of these product terms

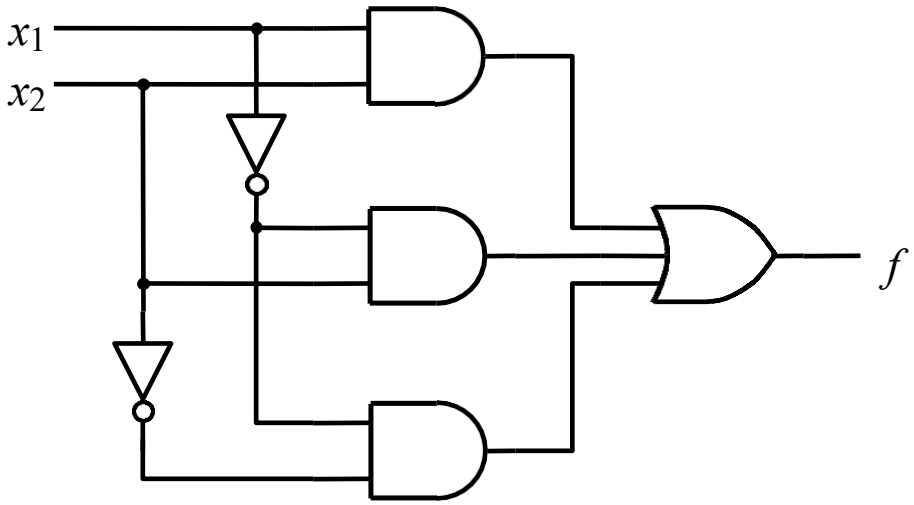
- Example: design a logic circuit with two inputs x_1 and x_2 and that the output of the circuit is 0 if switch x_1 is closed and switch x_2 is open; otherwise, the output must be 1. The truth table below expresses this required behavior

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

Synthesis using AND, OR, and NOT (Cont'd)

- Example (cont'd):

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1



Canonical sum-of-products

Logic function

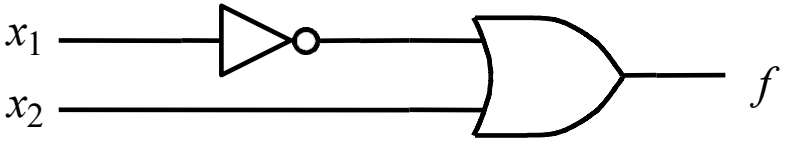
$f(x_1, x_2) = \overline{X1}\,\overline{X2} + \overline{X1}X2 + X1X2$

This is the operation implemented by 1st row of the truth table

This is the operation implemented by 2nd row of the truth table

This is the operation implemented by 4th row of the truth table

Can you make it simpler?



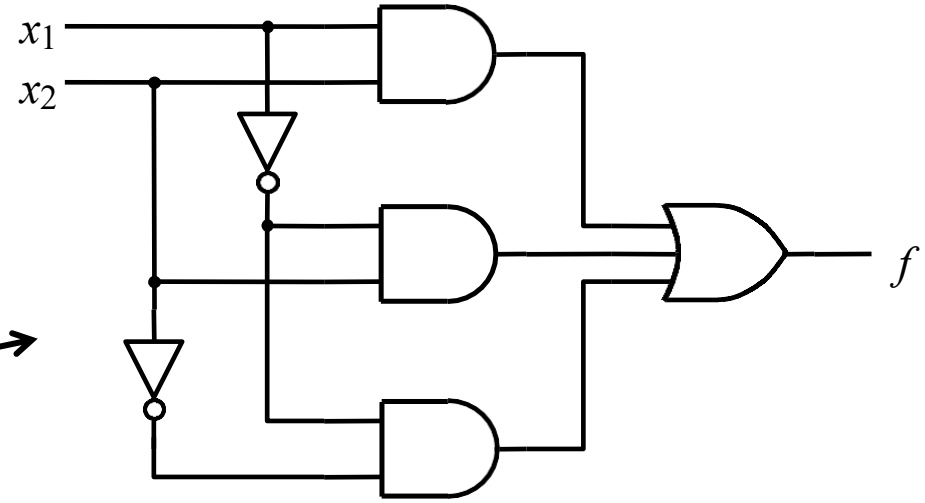
Minimal-cost realization

(Note that the method of synthesizing logic functions will be explained in more detail later from slide 39 when we discuss SOP form)

Making design simpler

From previous slide, we found that the logic function is given by:

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$



Canonical sum-of-products

We can use Boolean algebra to simplify this logic function.

Property 7b $f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2 + \bar{x}_1x_2$

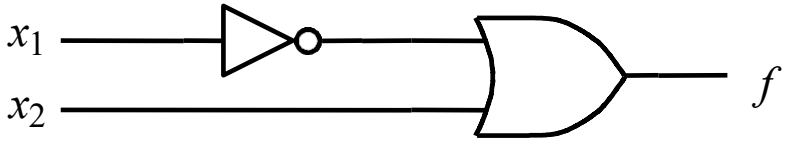
Property 10b $f(x_1, x_2) = x_1x_2 + \bar{x}_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$

Property 12a $f(x_1, x_2) = (x_1 + \bar{x}_1)x_2 + \bar{x}_1(\bar{x}_2 + x_2)$

Theorem 8b $f(x_1, x_2) = 1 \cdot x_2 + \bar{x}_1 \cdot 1$

Theorem 6a $f(x_1, x_2) = x_2 + \bar{x}_1$

Can you make it simpler?



Minimal-cost realization

Synthesis using AND, OR, and NOT (Cont'd)

- What is the best realization?
  - Reduce number of gates ➔ simpler and smaller circuits
  - Reduce number of input variables (literals)
    - Fewer input variables means faster gates, and less transistors
    - Number of gate inputs (fan-in) is limited in different technologies
  - Reduce propagation delay
    - Reduce number of gate levels
  - Be aware of hazards (glitches): static or dynamic
  - Consider the cost of implementation technology

Synthesis using AND, OR, and NOT (Cont'd)

- Formal definitions:
  - **Minterm:** for a function of n variables, a minterm is a product (AND) term in which each of the n variables (complemented or uncomplemented) appears once (**AND**ed product of literals)
  - **Maxterm:** is the complement (opposite) of minterm - for a function of n variables, a maxterm is a sum (OR) term in which each of the n variables (complemented or uncomplemented) appears once (**OR**ed sum of literals)

Row number	x_1	x_2	x_3	Minterm	Maxterm
0	0	0	0	$m_0 = \overline{x}_1\overline{x}_2\overline{x}_3$	$M_0 = x_1 + x_2 + x_3$
1	0	0	1	$m_1 = \overline{x}_1\overline{x}_2x_3$	$M_1 = x_1 + x_2 + \overline{x}_3$
2	0	1	0	$m_2 = \overline{x}_1x_2\overline{x}_3$	$M_2 = x_1 + \overline{x}_2 + x_3$
3	0	1	1	$m_3 = \overline{x}_1x_2x_3$	$M_3 = x_1 + \overline{x}_2 + \overline{x}_3$
4	1	0	0	$m_4 = x_1\overline{x}_2\overline{x}_3$	$M_4 = \overline{x}_1 + x_2 + x_3$
5	1	0	1	$m_5 = x_1\overline{x}_2x_3$	$M_5 = \overline{x}_1 + x_2 + \overline{x}_3$
6	1	1	0	$m_6 = x_1x_2\overline{x}_3$	$M_6 = \overline{x}_1 + \overline{x}_2 + x_3$
7	1	1	1	$m_7 = x_1x_2x_3$	$M_7 = \overline{x}_1 + \overline{x}_2 + \overline{x}_3$

$$\overline{m}_i = M_i$$

Synthesis using AND, OR, and NOT (Cont'd)

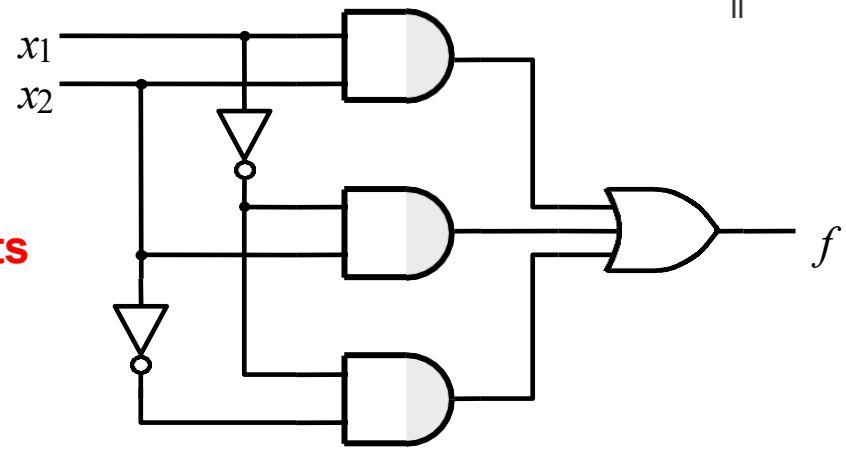
- Canonical forms: standard forms for a Boolean expression using truth table, and principle of duality.
  - **Sum-of-Product (SOP) form** - a function can be represented by an expression that is a sum of product (AND) terms, corresponding to the rows in the truth table for which **f = 1**. If **each product term is a minterm**, the expression is called canonical SOP.

- SOP example:

$$f = m_0 \bullet 1 + m_1 \bullet 1 + m_2 \bullet 0 + m_3 \bullet 1$$
$$= m_0 + m_1 + m_3 = \Sigma m(0,1,3)$$
$$= \overline{x}_1\overline{x}_2 + \overline{x}_1 x_2 + x_1 x_2$$

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

Canonical sum-of-products



Synthesis using AND, OR, and NOT (Cont'd)

- **SOP example:** first derive the canonical SOP expression for a given function, and then find a functionally equivalent SOP expression that has a lower cost.

$$f(x_1, x_2, x_3) = \sum(m_1, m_4, m_5, m_6)$$

$$f(x_1, x_2, x_3) = \sum m(1,4,5,6)$$

$$f(x_1, x_2, x_3) = m_1 + m_4 + m_5 + m_6$$

$$= \bar{x}_1 \bar{x}_2 x_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 \bar{x}_2 x_3 + x_1 x_2 \bar{x}_3$$

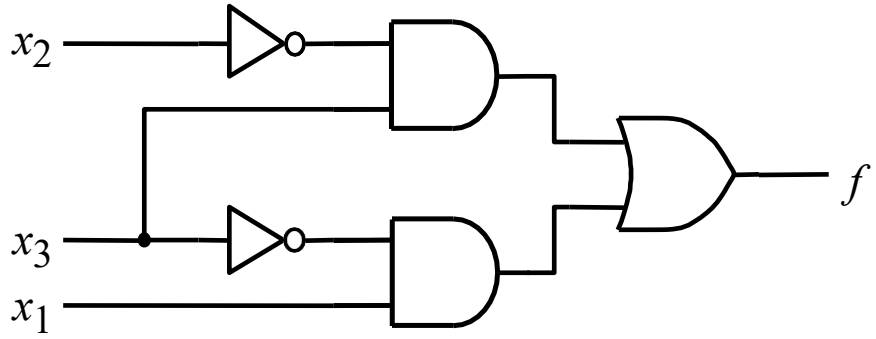
$$= (\bar{x}_1 + x_1)\bar{x}_2 x_3 + x_1(\bar{x}_2 + x_2)\bar{x}_3$$

$$= 1 \cdot \bar{x}_2 x_3 + x_1 \cdot 1 \cdot \bar{x}_3$$

$$= \bar{x}_2 x_3 + x_1 \bar{x}_3$$

Total cost = 5 (number of gates) + 8 (number of inputs to the gates) = 13;
Compare it with 17 of the canonical form.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0



A minimal sum-of-products realization

Example 2.10 from book

Example 2.10

Consider the function

$$f(x_1, x_2, x_3) = \sum m(2, 3, 4, 6, 7)$$

The canonical SOP expression for the function is derived using minterms

$$\begin{aligned} f &= m_2 + m_3 + m_4 + m_6 + m_7 \\ &= \bar{x}_1 x_2 \bar{x}_3 + \bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 \bar{x}_3 + x_1 x_2 x_3 \end{aligned}$$

This expression can be simplified using the identities in Section 2.5 as follows

$$\begin{aligned} f &= \bar{x}_1 x_2 (\bar{x}_3 + x_3) + x_1 (\bar{x}_2 + x_2) \bar{x}_3 + x_1 x_2 (\bar{x}_3 + x_3) \\ &= \bar{x}_1 x_2 + x_1 \bar{x}_3 + x_1 x_2 \\ &= (\bar{x}_1 + x_1) x_2 + x_1 \bar{x}_3 \\ &= x_2 + x_1 \bar{x}_3 \end{aligned}$$

Example 2.11 from book

Example 2.11

Suppose that a four-variable function is defined by

$$f(x_1, x_2, x_3, x_4) = \sum m(3, 7, 9, 12, 13, 14, 15)$$

The canonical SOP expression for this function is

$$f = \bar{x}_1\bar{x}_2x_3x_4 + \bar{x}_1x_2x_3x_4 + x_1\bar{x}_2\bar{x}_3x_4 + x_1x_2\bar{x}_3\bar{x}_4 + x_1x_2\bar{x}_3x_4 + x_1x_2x_3\bar{x}_4 + x_1x_2x_3x_4$$

A simpler SOP expression can be obtained as follows

$$\begin{aligned} f &= \bar{x}_1(\bar{x}_2 + x_2)x_3x_4 + x_1(\bar{x}_2 + x_2)\bar{x}_3x_4 + x_1x_2\bar{x}_3(\bar{x}_4 + x_4) + x_1x_2x_3(\bar{x}_4 + x_4) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2\bar{x}_3 + x_1x_2x_3 \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2(\bar{x}_3 + x_3) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2 \end{aligned}$$

Synthesis using AND, OR, and NOT (Cont'd)

- **Product-of-Sums (POS) form** – a function can be represented by an expression that is a product of sums (OR) terms, corresponding to the rows in the truth table for which $f = 0$. If **each sum term is a maxterm**, the expression is called canonical POS.

- POS example:

$$f(x_1, x_2) = m_0 + m_1 + m_3 \longleftarrow$$

POS expression which in the next steps is converted to the SOP expression

$$\overline{f}(x_1, x_2) = m_2 = x_1\overline{x_2}$$

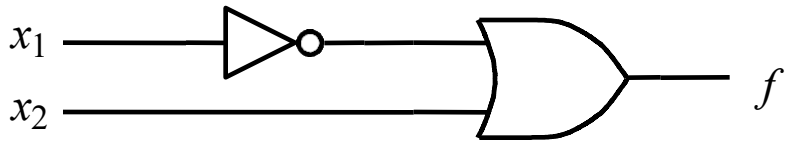
$$\overline{\overline{f}}(x_1, x_2) = f = \overline{x_1\overline{x_2}} = \overline{x_1} + x_2 = M_2$$

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

$$f = m_0 + m_1 + m_3 = \overline{m_2} = M_2$$

SOP form: Choose minterms for which $f = 1$

POS form: Choose maxterms for which $f = 0$



Canonical and minimal product-of-sums

Compare this with the figure on page 36.

Synthesis using AND, OR, and NOT (Cont'd)

- **POS example**: first derive the canonical POS expression for a given function, and then find a functionally equivalent POS expression that has a lower cost.

$$\overline{f}(x_1, x_2, x_3) = m_0 + m_2 + m_3 + m_7$$

$$f = \overline{m_0 + m_2 + m_3 + m_7}$$

$$f = \overline{m}_0 . \overline{m}_2 . \overline{m}_3 . \overline{m}_7$$

POS form

$$f = M_0 M_2 M_3 M_7 = \prod(M0, M2, M3, M7)$$

$$f = \prod M(0,2,3,7)$$

$$f = (x_1 + x_2 + x_3)(x_1 + \overline{x}_2 + x_3)(x_1 + \overline{x}_2 + \overline{x}_3)(\overline{x}_1 + \overline{x}_2 + \overline{x}_3)$$

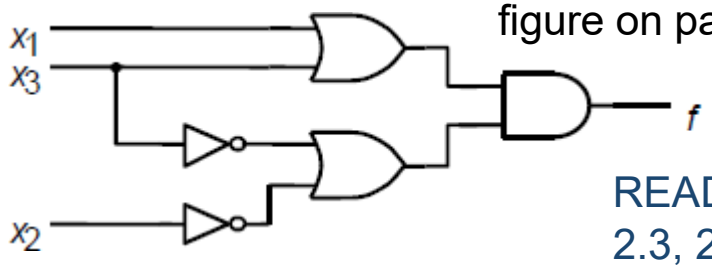
A canonical product-of-sums realization

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

➔ $$f = (x_1 + x_3)(\overline{x}_2 + \overline{x}_3)$$

A minimal product-of-sums realization

Compare this with the figure on page 40



READ the examples 2.3, 2.4

Example 2.12 from book

Example 2.12

Consider again the function in Example 2.10. Instead of using the minterms, we can specify this function as a product of maxterms for which $f = 0$, namely

$$f(x_1, x_2, x_3) = \Pi M(0, 1, 5)$$

Then, the canonical POS expression is derived as

$$\begin{aligned} f &= M_0 \cdot M_1 \cdot M_5 \\ &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3) \end{aligned}$$

A simplified POS expression can be derived as

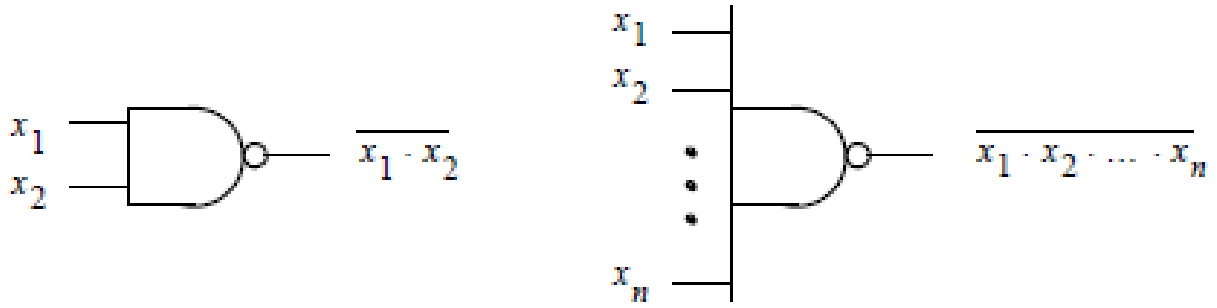
$$\begin{aligned} f &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3) \\ &= ((x_1 + x_2) + x_3)((x_1 + x_2) + \bar{x}_3)(x_1 + (x_2 + \bar{x}_3))(\bar{x}_1 + (x_2 + \bar{x}_3)) \\ &= ((x_1 + x_2) + x_3\bar{x}_3)(x_1\bar{x}_1 + (x_2 + \bar{x}_3)) \\ &= (x_1 + x_2)(x_2 + \bar{x}_3) \end{aligned}$$

The real difference between SOP and POS

- Normally SOP is used for Transistor to Transistor Logic (TTL)
  - NAND gates are the basis for TTL, so they fit well
    - As we will see shortly
- POS fits well with CMOS
  - All sizes of NOR gates are supported, so you can get a better result occasionally using POS with CMOS
- Ultimately the best choice also depends on complexity!
  - Complexity is a consideration of the number of gates, the fan-in and the implementation technology
- Example Time!

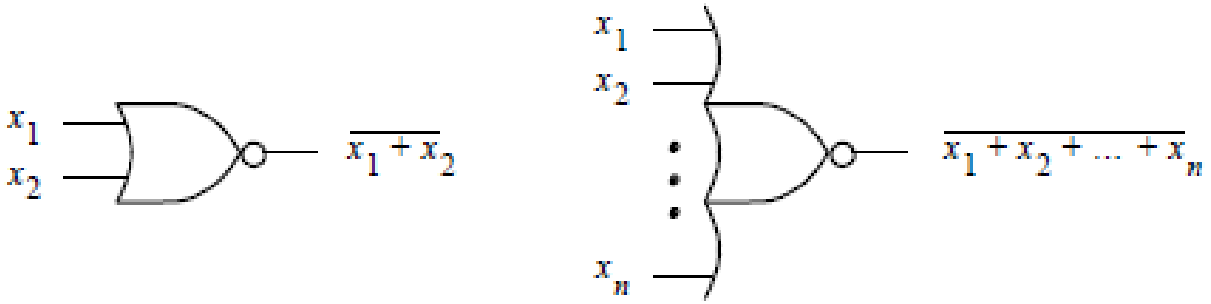
NAND and NOR Logic Networks

- NAND and NOR gates are attractive:
  - Simpler electronic circuits than AND and OR (to be discussed in chapter 3)



Truth table?

NAND gates

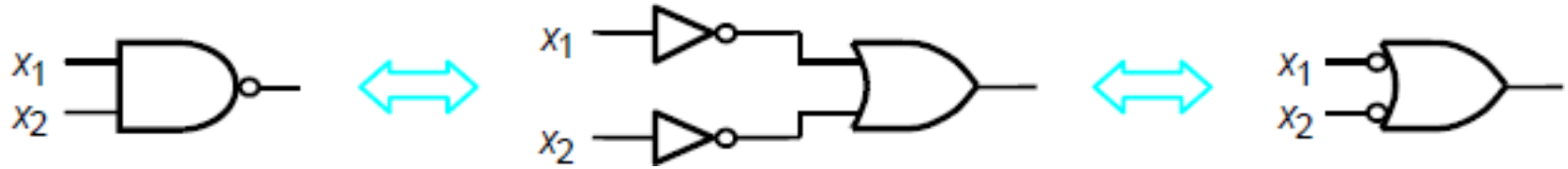


Truth table?

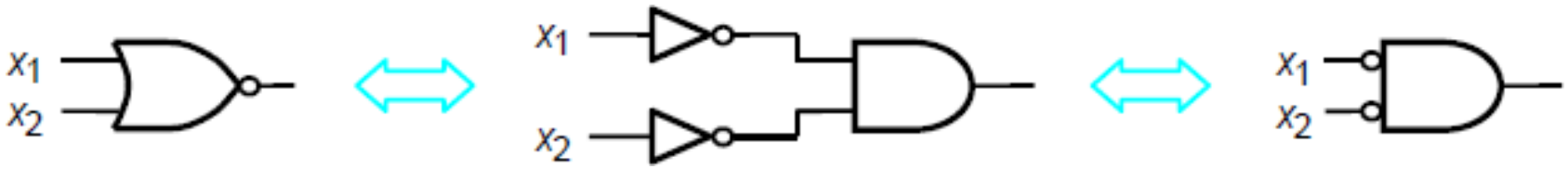
NOR gates

NAND and NOR Logic Networks (Cont'd)

- Can we use NAND and NOR gates in realizing the circuits? YES



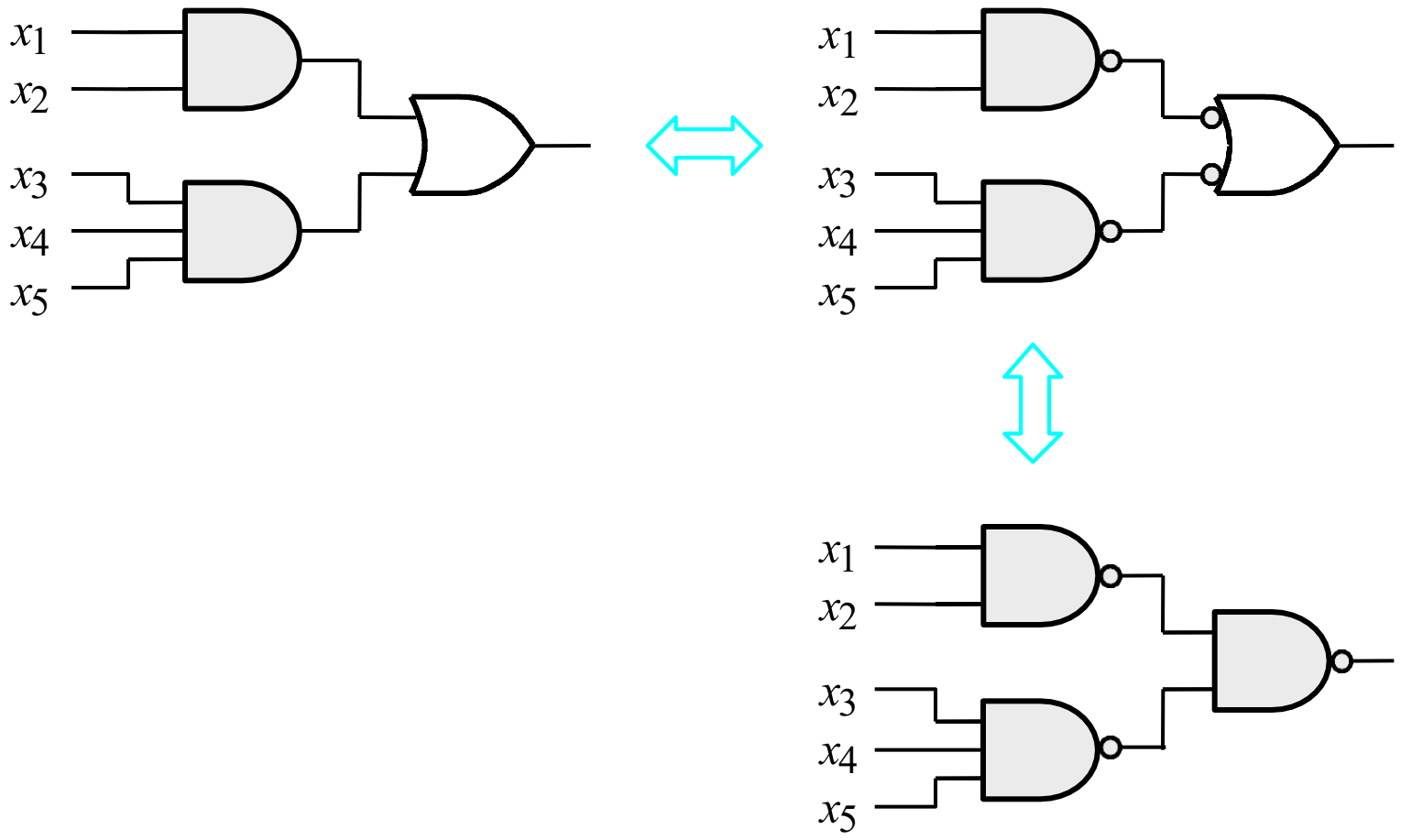
Use DeMorgan's theorem $\overline{x_1 x_2} = \bar{x}_1 + \bar{x}_2$



Use DeMorgan's theorem $\overline{x_1 + x_2} = \bar{x}_1 \bar{x}_2$

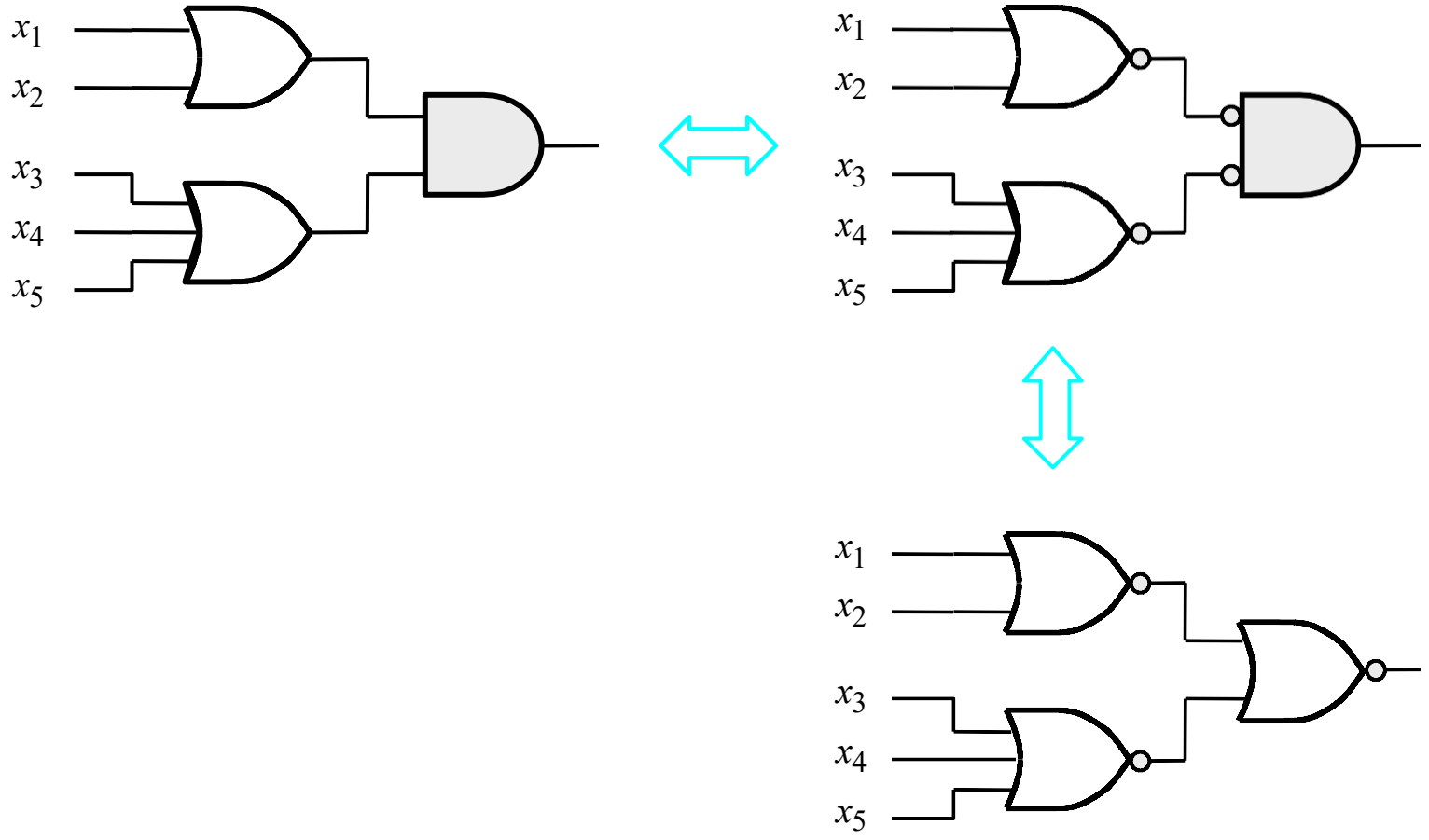
NAND and NOR Logic Networks (Cont'd)

- SOP to NAND-NAND network:



NAND and NOR Logic Networks (Cont'd)

- POS to NOR-NOR network:



NAND and NOR Logic Networks (Cont'd)

Example 2.13 Let us implement the function

$$f(x_1, x_2, x_3) = \sum m(2, 3, 4, 6, 7)$$

using NOR gates only. In Example 2.12 we showed that the function can be represented by the POS expression

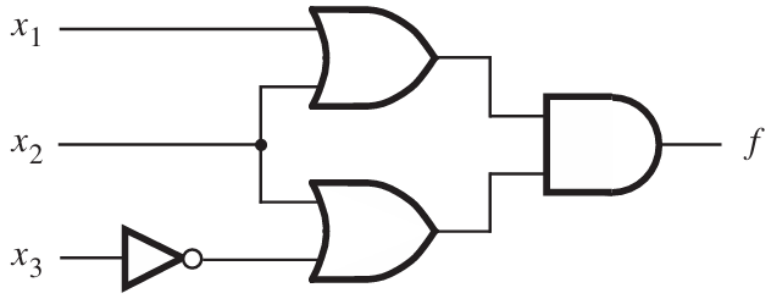
$$f = (x_1 + x_2)(x_2 + \bar{x}_3)$$

$f(x_1, x_2, x_3) = \sum m(2,3,4,6,7)$

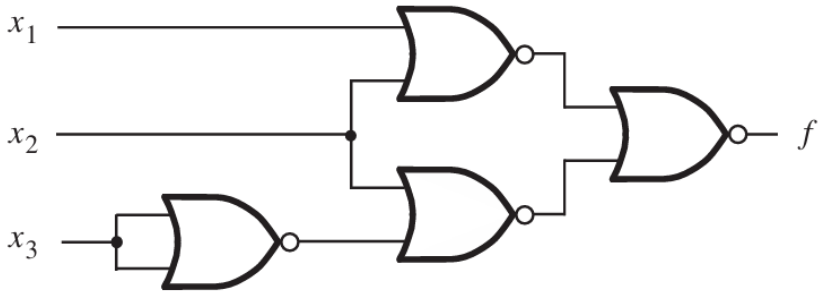
$f = \prod M(0,1,5) = (x_1 + x_2)(x_2 + \bar{x}_3)$

See example 2.12 (slide 45) for derivation

An OR-AND circuit that corresponds to this expression is shown in Figure 2.29*a*. Using the same structure of the circuit, a NOR-gate version is given in Figure 2.29*b*. Note that x_3 is inverted by a NOR gate that has its inputs tied together.



(a) POS implementation



(b) NOR implementation

Figure 2.29 NOR-gate realization of the function in Example 2.13.

Design Examples

- The desired behaviour of a digital circuit is first described in words, then it is turned into a formal specification for synthesis. The implementation then needs to be verified for the proper operation.

- Example: Three-way Light Controller - assume a large room with three doors, and a switch near each door that controls a light. The idea is to turn the light on and off using the switches, according to the following behaviour:

  - The light is off if all switches (x1, x2, and x3) are open.
  - The light is on if any one of the switches is closed.
  - The light will be off if two (or no) switches are closed.
  - The light will be on if all switches are closed.

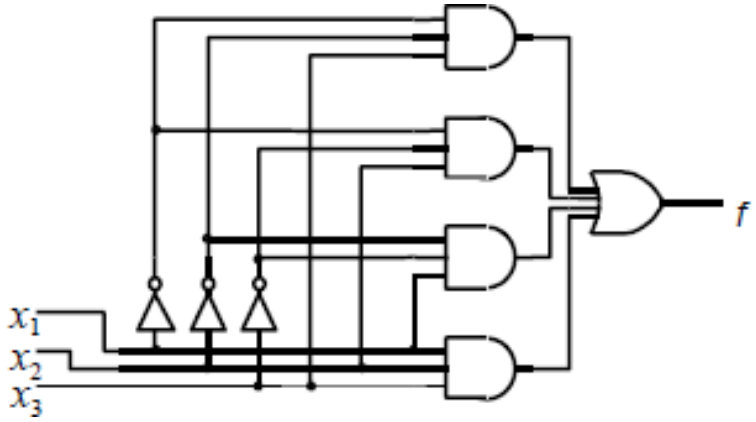
- Turn this functional behaviour into a truth table.

Design Examples (Cont'd)

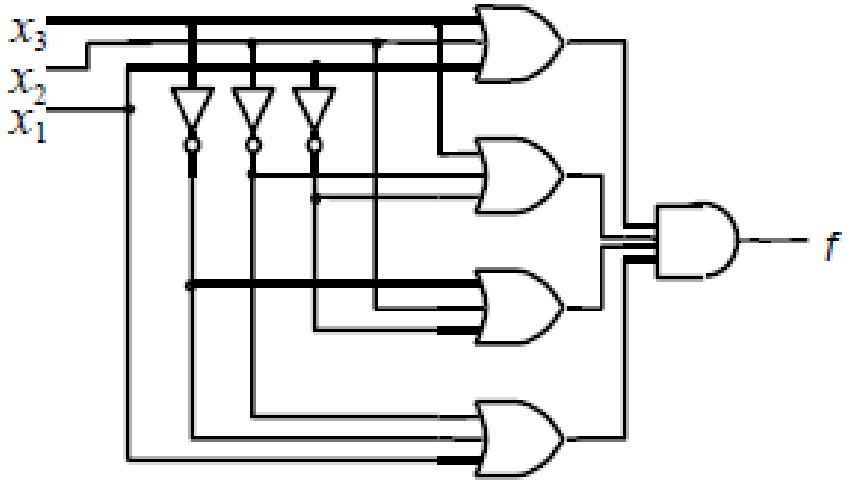
- Here is the truth table:

x_1	x_2	x_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Sum-of-products realization



Product-of-sums realization



$$f(x_1, x_2, x_3) = m_1 + m_2 + m_4 + m_7$$
$$f = \bar{x}_1 \bar{x}_2 x_3 + \bar{x}_1 x_2 \bar{x}_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 x_3$$

$$f = M_0 M_3 M_5 M_6$$
$$= (x_1 + x_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + x_3)$$

Design Examples (Cont'd)

- **Example: Multiplexer Circuit** – is a circuit that chooses data from exactly one of the number of possible input sources. It multiplexes input signals onto a single output signal.
  - Two sources of data (x1, x2), and an output f
  - A select input control signal s
  - If s = 0 ➔ f = x1 - If s = 1 ➔ f = x2

Truth table

$s\ x_1\ x_2$	$f(s, x_1, x_2)$
0 0 0	0
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	1

Sum-of-products realization

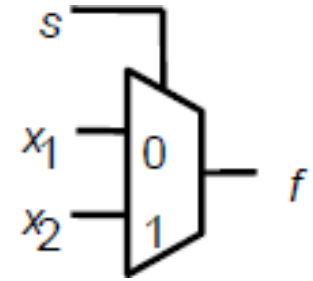
$$f = \bar{s}x_1\bar{x}_2 + \bar{s}x_1x_2 + s\bar{x}_1x_2 + sx_1x_2$$

$$f = \bar{s}x_1(\bar{x}_2 + x_2) + s(\bar{x}_1 + x_1)x_2$$

$$f = \bar{s}x_1 \cdot 1 + s \cdot 1 \cdot x_2$$

$$\boxed{f = \bar{s}x_1 + sx_2}$$

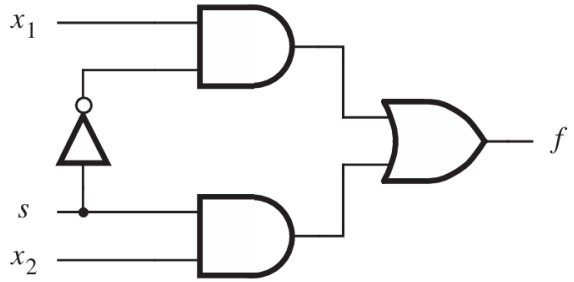
2-to-1 multiplexer



Graphical symbol

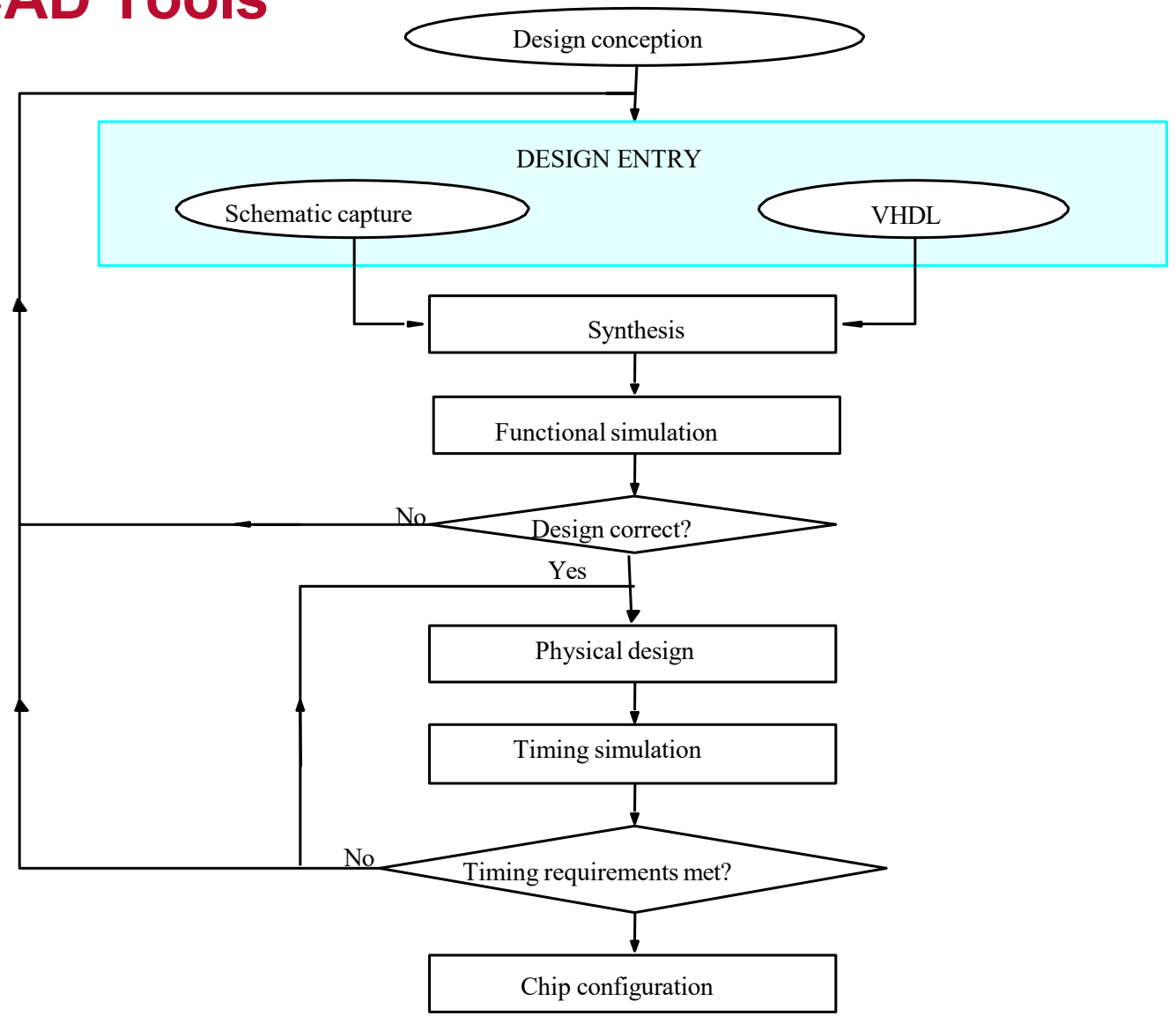
Compact truth table

s	$f(s, x_1, x_2)$
0	x_1
1	x_2



Circuit

Introduction to CAD Tools



A typical CAD system

Introduction to VHDL

- VHDL (Very High Speed Integrated Circuit Hardware Description Language) was designed by IBM, Texas Instruments, and Intermetrics. It was standardized by IEEE in 1987 (IEEE 1076-1987), and then enhanced in 1993 (IEEE 1076-1993), and updated in 2000, 2002 and 2018.
  - Additional packages include IEEE 1164 (data types), IEEE 1076.3 (numeric), and IEEE 1076.4 (timing).
- Using VHDL, one can describe the interface, and the functional (behavioral and structural) aspects of a digital system.
- Interface: the Entity construct

bit: 1 or 0

```
ENTITY example-1 IS
        PORT ( x1, x2, x3          : IN  BIT ;
               f                   : OUT BIT ) ;
END example-1 ;
```

Type bit is not powerful enough. Use IEEE 1164 9-value value system (not part of VHDL).

Introduction to VHDL (Cont'd)

- Behavioral description:

Signal assignment

```
ARCHITECTURE LogicFunc OF example-1 IS
BEGIN
        f <= (x1 AND x2) OR (NOT x2 AND x3) ;
END LogicFunc ;
```

- Another example:

```
entity example-2 IS
        port (x1, x2, x3, x4:        in bit;
              f, g:                  out bit);
end entity example-2;

architecture LogicFunc OF example-2 IS
begin
        f <= (x1 AND x3) OR (x2 AND x4);
        g <= (x1 OR NOT x3) AND (NOT x2 OR x4);
end LogicFunc;
```

Concurrent signal assignment

Read all the examples
at the end of this chapter

Introduction to VHDL (cont'd)

Last Example: VHDL for a 2-to-1 Multiplexer

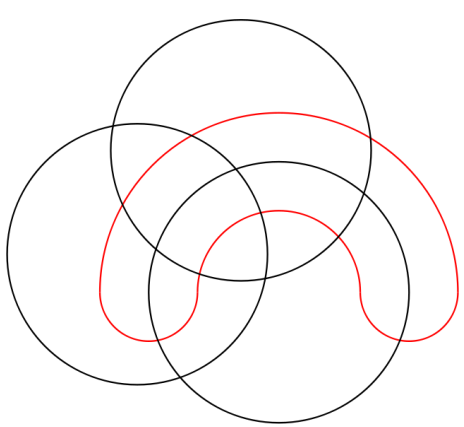
```
entity 2to1Mux IS
        port (s0, x0, x1:       in bit;
        f:                      out bit);
end 2to1Mux;

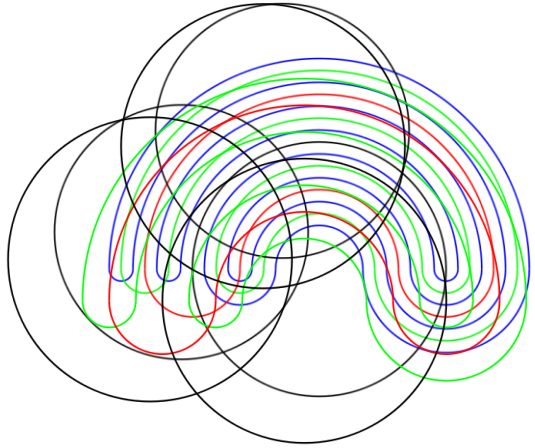
architecture Behaviour OF 2to1Mux IS
begin
        f <= x1 when (s0 = '1') else x0;
end 2to1Mux;
```

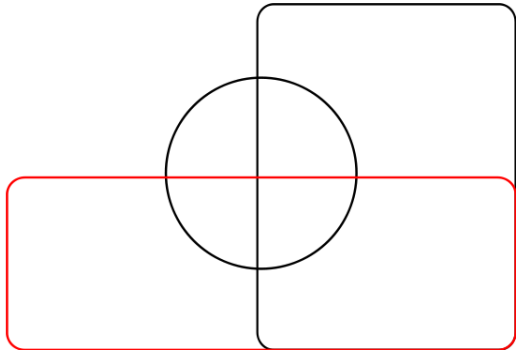
VHDL

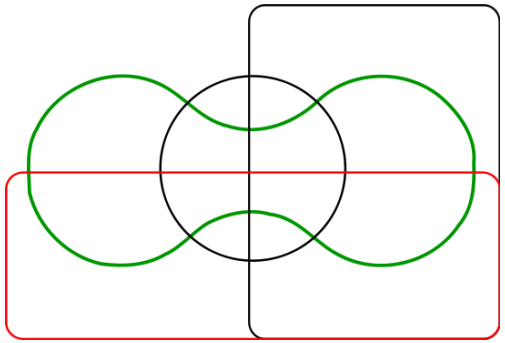
- VHDL looks a lot like a programming language, but its not, it has key differences
- The most important difference to note is that the language is not sequential
- Operations do not occur one after the other in the order they appear, but are resolved simultaneously through the described logic
- If..then type statements are only possible when put inside a special construct the PROCESS construct
- A Process statement indicates that the VHDL within that process statement should be synthesized as something that enforces ordering

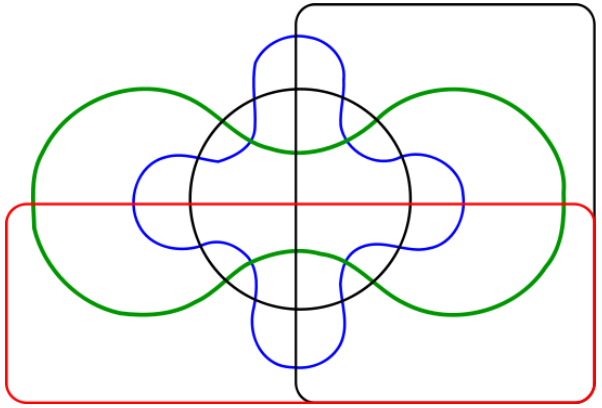
Backup: Venn Diagram with more than 3 variables

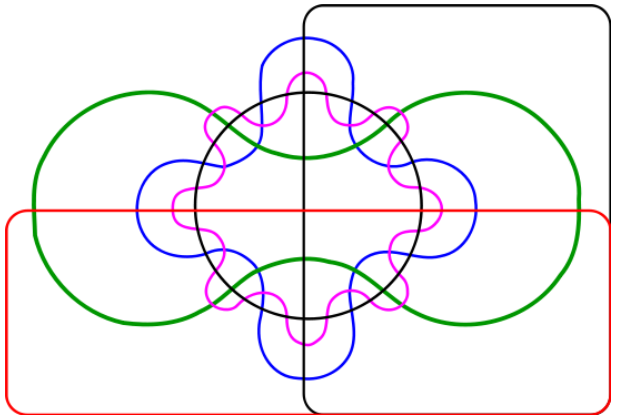














Queen's
UNIVERSITY